

Generators and Continuations

CSCI 1101

2024-05-03

Function Call Execution (simplified version)

Recall what happens when we call a function in Python:

1. The function argument expressions are evaluated.
2. Their values are assigned to the function parameter variables.
3. Control passes to the beginning of the function body.
4. Control flows within the function body until a return statement is reached.
5. The expression following the keyword `return` is evaluated to a return value.
6. The function call is abandoned and its variables are forgotten.
7. The return value becomes the value of the function call.
8. Control returns to place where the function was called.

Functions That Don't Return

In fact, function behavior can be richer than this model suggests.

A function does not *have to* return a value to its caller and then just disappear.

Here are two other possibilities for function behavior:

- A function can stick around after computing a result, then later we can call it again to pick off where it left off to compute more results.
- Instead of just returning its result to its caller, a function can decide for itself what to do with its result.

Generator Functions

A function that sticks around after computing a result is called a **generator**.

Generators are distinguished from ordinary functions in that they can `yield` as well as `return` a result.

Returning a result signals that the function's work is finished and it is not needed anymore.

Yielding a result signals that the function potentially has more work to do and can be called again, picking up where it left off.

Defining Generators

We define a generator just like any other function using the keyword `def`.

But instead of `returning` a result, we `yield` it:

```
def my_range(stop) :  
    num = 0  
    while num < stop :  
        yield num    # can pick up where we left off  
        num += 1
```

Generating Results

Calling a *generator function* creates a **generator object**:

```
r = my_range(3)
```

To get a generator to *yield* a result we call the `next` function:

```
> next(r)
```

```
0
```

```
> next(r)
```

```
1
```

```
> next(r)
```

```
2
```

If a generator runs out of results to *yield* it raises a `StopIteration` exception.

Generators are Iterators

Generator objects are iterators.

This means that we can use them in a `for` loop:

```
for num in my_range(3) :  
    print(num)
```

The loop will iterate once for each result `yielded` by the generator.

In fact, this is how many of the iterators that we met in this course are defined.

Activity: Generators

Write a generator called `plus_minus` that takes an `int` argument and `yields` the sequence of alternating positive and negative numbers starting from 1 whose absolute value is less than the limit.

```
def plus_minus(stop) :
```

For example:

```
> list(plus_minus(4))  
[1, -1, 2, -2, 3, -3]
```


Continuations

When a function is called and its result returned, the caller can decide what to do with it.

The rest of the program after a function call is known as its **continuation**.

For the function

```
def add(x , y) :  
    ''' sig: (int , int) --> int '''  
    return x + y
```

in the program

```
> num = add(1 , 2)  
> print(num)  
3
```

Its continuation is to print the value of the function call.

Continuation Passing Style

We can rewrite the add function to take an additional argument representing its continuation.

This way instead of the caller of the function taking care of what to do with the result, the function will do so itself by calling its continuation:

```
def add_c(x , y , cont) :  
    ''' sig: (int , int , fun (int -> a)) --> a '''  
    return cont(x + y)
```

For example:

```
> add_c(1 , 2 , print)  
3  
> add_c(1 , 2 , lambda n : 2 * n)  
6
```

Choose Your Own Continuation

A function can take multiple continuations and pick which one to call conditionally:

```
def parity_c(num , even_cont , odd_cont) :  
    ''' sig: (int , fun (int -> a) , fun (int -> a)) --> a '''  
    return even_cont(num) if num % 2 == 0 else odd_cont(num)
```

For example:

```
> parity_c(3 , lambda n: n//2 , lambda n: n+1)  
4  
> parity_c(4 , lambda n: n//2 , lambda n: n+1)  
2
```

Discarding Continuations

A function can even decide not to call its continuation:

```
def divide_c(x , y , cont) :  
    if y != 0 :  
        return cont(x / y)  
    else :  
        print("Can't divide by 0!")
```

For example:

```
> divide_c(4 , 2 , print)  
2.0  
> divide_c(4 , 0 , print)  
Can't divide by 0!
```

This is the basis on which features like exceptions are implemented.

Summary

More advanced aspects of functions such as generators and continuations enable programming language features like coroutines and exceptions.

It's good to remain open minded about what programming can be.

There's always more to learn.

To Do

Finish *Project 4: Bioinformatics* on CodeRunner in by 2024-05-08.